

Inlämningsuppgift: Säker och integrerad IoT-lösning

Bakgrund

Ett fastighetsbolag vill samla in mätvärden från sensorer i en byggnad. Informationen ska överföras genom nätverket, göras tillgänglig via ett API och kunna användas av andra system.

Lösningen behöver vara dokumenterad, möjlig att övervaka och robust mot vanliga kommunikationsfel. Företaget vill också att säkerhetsrisker och tekniska begränsningar redovisas.

Uppgift

Utveckla och dokumentera en nätverksbaserad IoT-lösning som:

1. Tar emot data från minst en IoT-enhet
2. Överför data med valt nätverksprotokoll, motivera valet. T.ex. MQTT
3. Behandlar strukturerad data i JSON eller XML
4. Tillgängliggör eller konsumerar data genom ett API
5. Använder grundläggande säker kommunikation
6. Har loggning och minst en funktion för övervakning
7. Hanterar minst två avsiktligt införda fel
8. Dokumenteras så att en annan utvecklare kan installera, köra och felsöka lösningen

ESP32 (C6 om ni lånar från skolan, egna enheter får förstås också användas) ska användas som fysisk IoT-enhet. Lösningen ska innehålla minst ett mätvärde som skapas av en fysisk sensor (alternativt något fysiskt för att fejka inkommande data - t.ex. en knapp eller potentiometer) ansluten till IoT-enheten. Externa API:er och simulerade sensorer får användas för att komplettera detta med ytterligare data om man vill.

Tekniska minimikrav

Lösningen ska innehålla följande delar. Att tänka på är att det är många delar, så det är *inte* tänkt att varje del ska vara jättestor/djupgående.

1. IoT-enhet och sensor

IoT-enheten ska samla in minst ett verkligt mätvärde från en ansluten sensor, exempelvis:

- Temperatur
- Luftfuktighet
- Ljusnivå
- Rörelse
- Energiförbrukning

Mätvärdet ska skickas periodiskt eller när en händelse inträffar. Data från externa API:er eller simulerade sensorer får användas som komplement, exempelvis för att representera fler sensorer eller tillföra jämförelsedata. Den kompletterande datan ersätter alltså *inte* det fysiska mätvärdet.

2. Nätverkskommunikation

Använd lämpligt protokoll, t.ex. MQTT. Följande ska dokumenteras:

- Använda IP-adresser eller värdnamn
- Portar
- Kommunikationsmodell
- Topics eller motsvarande adressering
- Hur återanslutning och kommunikationsfel hanteras
- Varför det valda protokollet passar lösningen

3. Strukturerad data

Sensorinformationen ska överföras som giltig JSON eller XML.

Exempel:

```
{
  "sensorId": "room-a-temp-01",
  "timestamp": "2026-09-17T10:30:00+02:00",
  "value": 21.7,
  "unit": "C"
}
```

Datakontraktet ska beskrivas och användas konsekvent.

4. API-integration

Lösningen ska antingen:

- Tillgängliggöra sensordata genom ett eget API
- Konsumera data från en extern webbtjänst
- Eller båda två

API-integrationen ska omfatta minst ett fungerande anrop eller en fungerande operation. HTTP-metod, URL, förväntat svar och felhantering ska dokumenteras.

5. Säkerhet

Tillämpa minst två relevanta säkerhetsåtgärder, exempelvis:

- TLS eller HTTPS
- Autentisering
- Behörighetskontroll
- Säker hantering av lösenord och tokens
- Nätverkssegmentering
- Validering av inkommande data
- Begränsad åtkomst genom brandväggsregler

Inga autentiseringsuppgifter eller andra hemligheter får sparas i versionshanteringen.

6. Loggning och övervakning

Lösningen ska logga relevanta händelser, exempelvis:

- Anslutning och återanslutning
- Mottagen eller skickad data
- API-anrop
- Valideringsfel
- Kommunikationsfel

Implementera även minst ett enkelt övervakningsmått, exempelvis antal meddelanden, felräknare, senaste mottagna mätvärde eller tid sedan senaste kontakt.

7. Felsökning

Lägg medvetet till och analysera minst två fel, exempelvis:

- Fel brokeradress eller port
- Fel MQTT-topic
- Ogiltig JSON eller XML
- Timeout från API
- Avbruten nätverksanslutning
- Felaktig autentisering
- Otillgänglig tjänst

För varje fel ska ni dokumentera:

1. Observerat symptom
2. Hur felet identifierades
3. Vilka verktyg eller loggar som användes
4. Felets orsak
5. Genomförd åtgärd
6. Hur ni verifierade att åtgärden fungerade

Tips! Be en klasskamrat lägga in ett fel i taget åt er så blir felsökningen mer riktig då ni inte har full koll på vad de ändrat.

Leverabler

Lämna in en länk till ett GitHub-repo som innehåller:

```
README.md
src/      // eller motsvarande där källkoden ligger (får förstås innehålla
undermappar för att strukturera upp den hur ni vill)
docs/
  arkitektur.md
  api.md
  sakerhet.md
  felsokning.md
tests/    // en test-mapp för enhets- och/eller integrationstester, eller
testprotokoll.md med end-to-end-tester
```

README

README ska innehålla:

- Lösningens syfte
- Installationsanvisningar
- Nödvändiga beroenden
- Konfiguration
- Instruktioner för att starta lösningen
- Instruktioner för att verifiera dataflödet
- Kända begränsningar

Exempel på READMEs

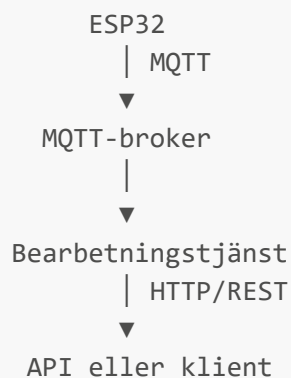
[README 101](#)

[Exempel 1](#)

[Lång lista på exempel](#)

Arkitekturbeskrivning

Dokumentera lösningens komponenter och kommunikationsvägar med ett diagram, exempelvis:



Ange protokoll, portar, dataformat och komponenternas ansvar.

API-dokumentation

Dokumentera:

- Endpoint och HTTP-metod
- Parametrar
- Exempel på anrop
- Exempel på svar
- Felkoder eller felkontrakt

Exempel

[REST-API-example](#) - Detta är visserligen deras README, men den har både exempel på hur man gör anrop för specifika saker och data som returneras. **OBS!** Ni behöver inte dokumentera lika mycket/noggrant. Det skulle

ta alltför lång tid från projektet.

Säkerhetsanalys

Beskriv:

- Minst tre relevanta risker
- Genomförda säkerhetsåtgärder för att minska eller hantera risker
- Kvarvarande begränsningar
- Hur känslig konfiguration (Wi-Fi SSID och lösenord, eventuella andra användarnamn och lösenord) hanteras

Redovisning

Eftermiddagen sista dagen (8/10 -26) kör vi en muntlig redovisning för skojs skull. Det är alltid kul att se hur andras lösningar ser ut, plus att det är bra att träna på. Redovisningen är dock **inte** ett examinationsmoment.

Varje person pratar lite kort om sitt projekt och visar:

- Systemets dataflöde
- API-integrationen
- En säkerhetsåtgärd
- Ett felsökningsfall
- En viktig teknisk lärdom

Bedömning

Inlämningsuppgiften bedöms med betygen IG, G eller VG. Bedömningen görs utifrån den inlämnade lösningen och dokumentationen. Redovisningen är *inte* en del av bedömningen.

Godkänt

För G ska lösningen uppfylla uppgiftens tekniska minimikrav. Det innebär att:

- Det finns ett fungerande dataflöde från IoT-enhet till övriga delar av systemet
- Nätverkskommunikation, API-integration, säkerhet, loggning och övervakning har implementerats
- Minst två fel har identifierats, analyserats och åtgärdats
- Kod och dokumentation gör det möjligt för någon annan att förstå, köra och felsöka lösningen

Väl godkänt

För VG ska samtliga krav för G vara uppfyllda. Dessutom ska arbetet visa hög teknisk precision och självständighet genom att:

- Systemets olika delar är väl integrerade och fungerar som en sammanhängande lösning
- Val av protokoll, API-integration och säkerhetsåtgärder motiveras utifrån lösningens krav
- Kommunikationsfel och andra störningar hanteras på ett genomtänkt och robust sätt
- Dokumentationen analyserar lösningens begränsningar och beskriver relevanta förbättringar

Icke godkänt

IG ges om lösningen eller dokumentationen har sådana brister att uppgiftens tekniska minimikrav inte är uppfyllda.